

Phishing Website Detection using Machine Learning

Silin Michael

Abstract

This project reproduces and critically evaluates a phishing website detection system based on engineered URL and website features. Three classical machine learning models—Logistic Regression, Decision Tree, and Random Forest—were trained and evaluated on a publicly available phishing dataset. The reproduction extends the original work by presenting a structured machine learning workflow, feature engineering analysis, reproducibility analysis, duplicate-row robustness testing, domain-aware evaluation, and threshold-based risk analysis. Among the evaluated models, Random Forest achieved the strongest performance on the regular random split, while the robustness and generalization analyses showed that duplicate rows and domain leakage can significantly affect performance estimates.

1 Executive Summary

This project critically evaluates and reproduces a public phishing website detection repository. The original source proposes using machine learning models trained on engineered URL and website-related features to classify websites as either legitimate or phishing.

The dataset used in this project is `5.urldata.csv`. Each row represents a website and includes engineered features such as URL length, URL depth, shortened URL usage, DNS record information, domain age, iframe behavior, and redirection behavior. The binary label indicates whether the website is legitimate or phishing.

This reproduction trained three classical machine learning models: Logistic Regression, Decision Tree, and Random Forest. On the regular random split, Random Forest achieved the strongest performance, with Accuracy 0.868, Precision 0.926, Recall 0.800, F1 Score 0.858, F2 Score 0.822, ROC-AUC 0.926, and PR-AUC 0.943.

The evaluation was extended beyond the original workflow by including duplicate-row analysis, domain-aware validation, and threshold analysis. Duplicate-row analysis showed that the dataset contains 5,626 duplicated rows. After removing duplicates, the dataset size dropped from 10,000 to 4,374 rows and became highly imbalanced. Domain-aware grouped splitting showed that Random Forest performance dropped from an F1 Score of 0.858 to 0.781 when the same domain was prevented from appearing in both the training and testing sets.

Threshold analysis demonstrated the trade-off between reducing false negatives and increasing false positives. In particular, a threshold of 0.3 provided a practical compromise by substantially reducing false negatives while avoiding the large increase in false positives observed at a threshold of 0.2.

Overall, the reproduction supports the original claim that engineered website features can detect phishing websites. At the same time, it shows that duplicate rows, domain leakage, and threshold choice must be considered before drawing conclusions about real-world performance.

2 Summary of the Source

The selected source for this project is the public GitHub repository *Phishing Website Detection*, available at:

`https://github.com/gangeshbaskerr/Phishing-Website-Detection`

The source addresses the problem of phishing website detection. Phishing websites imitate legitimate websites in order to steal sensitive information such as usernames, passwords, financial details, or personal data. This is an important cybersecurity problem because phishing attacks are common, easy to deploy, and can cause serious harm to individuals and organizations.

The proposed solution in the original repository is to use machine learning models trained on engineered website and URL-based features. Instead of analyzing raw web pages directly, the dataset contains extracted features that describe suspicious website behavior and URL structure. The main dataset file used in this reproduction is:

`data/5.urldata.csv`

Each row represents one website and contains a binary label:

- **0**: Legitimate website
- **1**: Phishing website

The features include URL-based properties, domain-related properties, and website behavior indicators. Examples include whether the URL contains an IP address, whether it uses a shortened URL, URL length, URL depth, redirection behavior, DNS records, domain age, iframe usage, mouse-over behavior, and right-click behavior.

The original repository demonstrates that classical machine learning models can be used to distinguish phishing websites from legitimate websites. This reproduction investigates whether that claim is supported by the data, whether the methodology is reproducible, and whether the evaluation is sufficiently rigorous.

3 Critical Evaluation

The main claim of the original repository is that engineered URL and website-related features can be used with machine learning models to detect phishing websites effectively. This reproduction supports this claim on the regular random train-test split, where Random Forest achieved the strongest overall performance.

However, the deeper evaluation revealed important limitations. First, the dataset contains many duplicated rows. The original dataset contains 10,000 rows, but 5,626 of them are exact duplicates. After removing duplicated rows, only 4,374 rows remain. This is a major data-quality issue because duplicate rows can make a random train-test split easier if identical or nearly identical records appear in both the training and testing sets.

Second, the original evaluation does not sufficiently address domain leakage. If the same domain appears in both the training set and the test set, the evaluation may overestimate the model’s ability to generalize to unseen websites. To evaluate this issue, a grouped split by `Domain` was used, ensuring that the same domain could not appear in both the training and testing data. Under this stricter grouped split, Random Forest performance dropped from F1 Score 0.858 to 0.781, showing that the original random split likely provides an optimistic estimate of generalization.

Third, the original work does not sufficiently discuss the asymmetric cost of errors in phishing detection. False negatives are especially dangerous because they represent phishing websites that were incorrectly classified as legitimate. For this reason, the reproduction includes F2 Score, PR-AUC, precision-recall curves, and threshold analysis to study whether reducing false negatives is worth the corresponding increase in false positives.

Overall, the original claim is supported within the limits of the dataset, but the reproduction shows that stronger evaluation is necessary. The original work would be improved by duplicate-aware experiments, grouped domain-based validation, threshold analysis, and a deeper discussion of false positives and false negatives.

4 Feature Engineering Analysis

Feature engineering is central to this project. The dataset does not use raw website content directly. Instead, it uses manually engineered features that describe properties of the URL, domain, and website behavior.

The `Domain` column was excluded from model training. This was done because domain names are textual identifiers and could cause the model to memorize specific websites instead of learning general phishing-related patterns. However, domain names were later used for grouped-split analysis and error interpretation.

The feature matrix X was created by removing the `Domain` column and the target

label column. The target vector y was created from the `Label` column.

Two preprocessing approaches were used:

- Tree-based models used the original unscaled numeric features.
- Logistic Regression used a `StandardScaler` preprocessing pipeline.

Scaling was used for Logistic Regression because the model is sensitive to feature magnitudes. Standardization transforms a feature x into:

$$z = \frac{x - \mu}{\sigma}$$

where μ is the mean and σ is the standard deviation. This helps Logistic Regression treat features on comparable scales. Tree-based models such as Decision Trees and Random Forests do not require scaling because they split data according to feature thresholds.

From a cybersecurity perspective, many of the engineered features are meaningful. For example, phishing websites often use long URLs, unusual URL depth, shortened links, suspicious redirections, or recently created domains. Features such as `URL_Length`, `URL_Depth`, `TinyURL`, `Domain_Age`, and `DNS_Record` therefore have practical security meaning.

From a mathematical perspective, possible redundancy can be detected using correlation analysis. If two features are highly correlated, they may provide overlapping information. In this project, Spearman correlation was used because many features are binary or ordinal rather than normally distributed continuous variables.

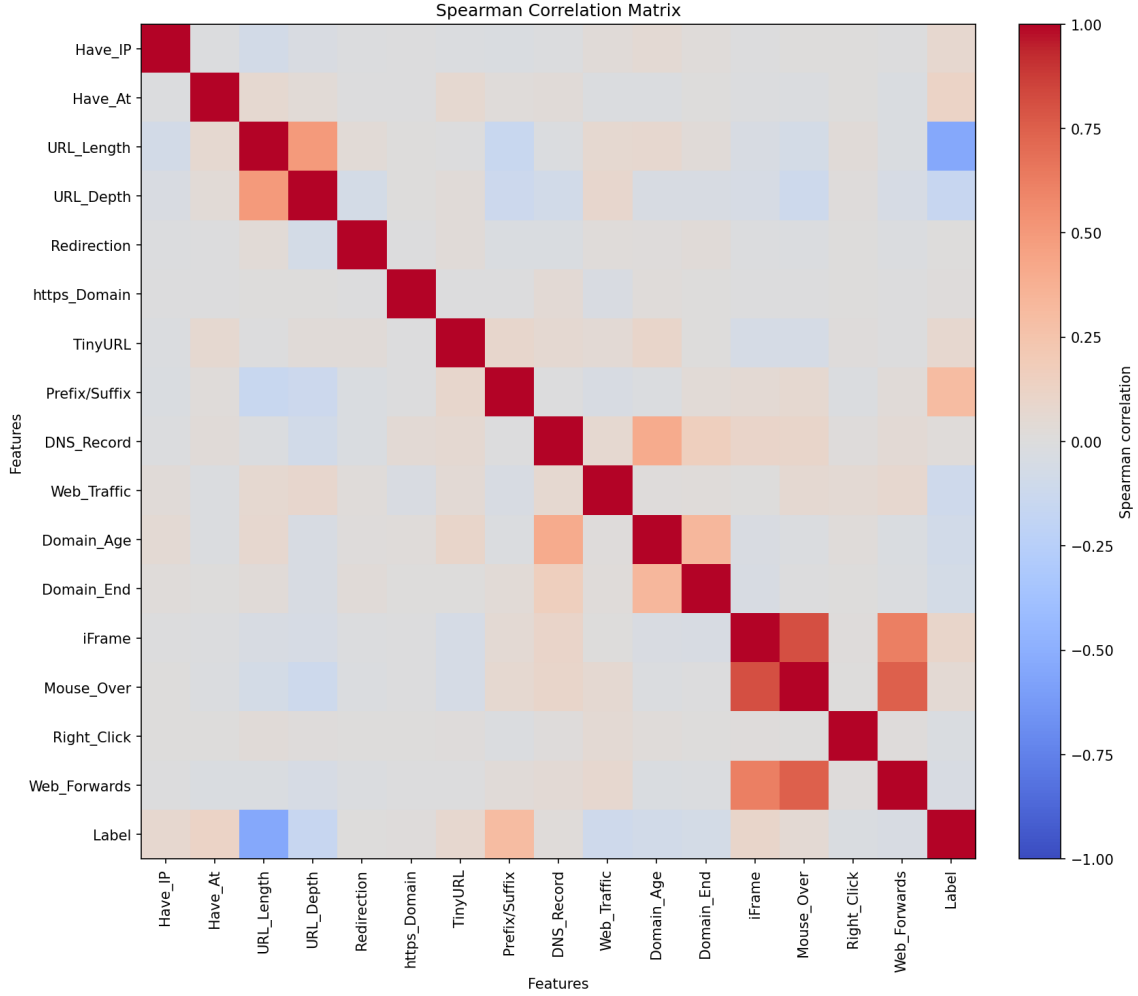


Figure 1: Spearman correlation matrix between engineered features.

Additional features that could improve performance include TLS certificate information, WHOIS registration details, domain reputation scores, character entropy of the URL, page content features, JavaScript behavior, login form detection, and blacklist or threat-intelligence indicators. These features could make the model more robust against modern phishing websites.

5 Reproducibility Analysis

The original repository provides the dataset and implementation, which makes reproduction possible. However, the workflow required several adjustments in order to make the project cleaner and more reproducible.

The dataset file `5.urldata.csv` is now included locally in the project under:

`data/5.urldata.csv`

The README also explains how to obtain the file from the original repository if it

is missing. This improves reproducibility because the notebook can now be run directly from the project root without relying on a sibling repository path.

The workflow was organized into a structured notebook that follows a complete machine learning pipeline. The notebook includes data loading, inspection, exploratory data analysis, preprocessing, model training, evaluation, duplicate-row analysis, grouped domain split analysis, feature importance analysis, error analysis, and threshold analysis.

The final reproduction focuses on Logistic Regression, Decision Tree, and Random Forest. These models can be executed using standard scikit-learn dependencies without requiring additional platform-specific libraries.

Overall, the original work was reproducible in the sense that the dataset and general approach were available. However, this reproduction improves reproducibility by including clearer dataset instructions, a local dataset path, fixed random seeds, a structured notebook, and a more complete evaluation workflow.

6 Experimental Results

6.1 Exploratory Data Analysis

Exploratory Data Analysis was performed before model training. The class distribution was examined to determine whether the dataset was balanced between phishing and legitimate websites.

A missing-value analysis was also performed before model training. No missing values were found in any feature column or in the target label, so no imputation or row removal was required.

No temporal feature is available in this dataset, so temporal trends or time-based splitting could not be analyzed.

Because most features are binary or small ordinal indicators, outlier analysis focused mainly on checking numeric ranges such as `URL_Depth`.

The dataset contains 10,000 rows, with 5,000 legitimate websites and 5,000 phishing websites. Therefore, the dataset is perfectly balanced in terms of class labels. However, 5,626 duplicated rows were found, meaning that duplicate samples are a major data-quality issue.

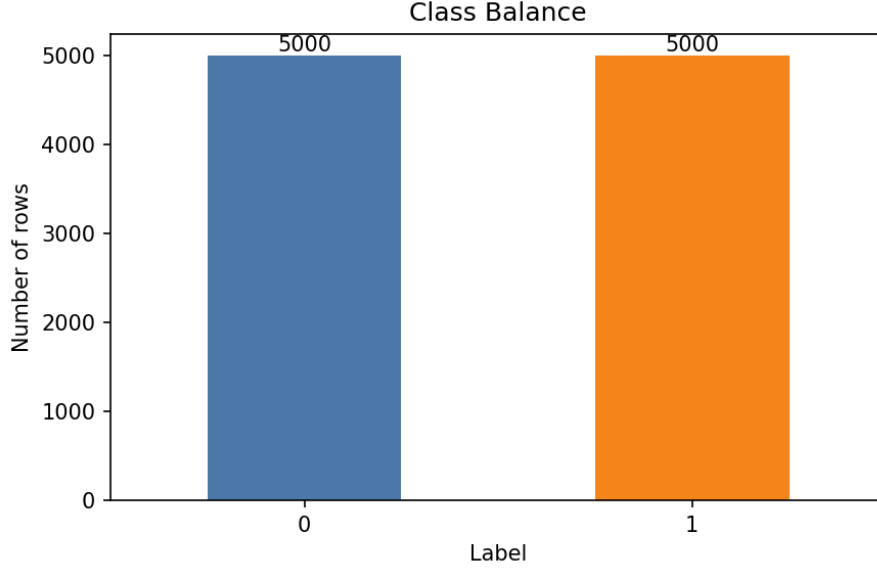


Figure 2: Class balance of legitimate and phishing websites.

6.2 Models Trained

Three classification models were trained and evaluated:

- **Logistic Regression:** A simple linear classification model used as a baseline.
- **Decision Tree:** A tree-based model that learns rule-like decisions from the data.
- **Random Forest:** An ensemble of decision trees that improves stability and generalization.

Each model was trained on the training set and evaluated on the held-out test set.

6.3 Evaluation Metrics

The models were evaluated using Accuracy, Precision, Recall, F1 Score, F2 Score, MCC, ROC-AUC, PR-AUC, and confusion matrices.

Let TP be true positives, TN true negatives, FP false positives, and FN false negatives.

Accuracy measures the overall proportion of correct predictions:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision measures how many websites predicted as phishing were actually phishing:

$$Precision = \frac{TP}{TP + FP}$$

Recall measures how many actual phishing websites were detected:

$$Recall = \frac{TP}{TP + FN}$$

F1 Score balances precision and recall equally:

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$$

F2 Score gives more weight to recall than precision:

$$F2 = \frac{5 \cdot Precision \cdot Recall}{4 \cdot Precision + Recall}$$

This is useful in phishing detection because missing a phishing website is often more dangerous than incorrectly flagging a legitimate website.

ROC-AUC measures the model’s ability to distinguish phishing websites from legitimate websites across all possible classification thresholds:

$$AUC = \int_0^1 TPR(FPR^{-1}(t)) dt$$

PR-AUC measures the area under the precision-recall curve:

$$PR-AUC = \int_0^1 Precision(Recall) dRecall$$

PR-AUC is especially useful when focusing on the positive phishing class.

The Matthews Correlation Coefficient (MCC) provides a balanced measure of classification quality:

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

MCC ranges from -1 to 1 , where 1 indicates perfect classification, 0 corresponds to random prediction, and -1 indicates complete disagreement between predictions and true labels.

6.4 Baseline Results

Table 1: Performance of the evaluated machine learning models on the regular random split.

Model	Accuracy	Precision	Recall	F1	F2	MCC	ROC-AUC	PR-AUC
Random Forest	0.868	0.926	0.800	0.858	0.822	0.743	0.926	0.943
Decision Tree	0.865	0.925	0.794	0.855	0.817	0.737	0.917	0.936
Logistic Regression	0.796	0.923	0.647	0.761	0.688	0.621	0.874	0.905

The Random Forest classifier achieved the strongest overall performance on the regular random split. It obtained an F1 Score of 0.858, F2 Score of 0.822, ROC-AUC of 0.926, and PR-AUC of 0.943.

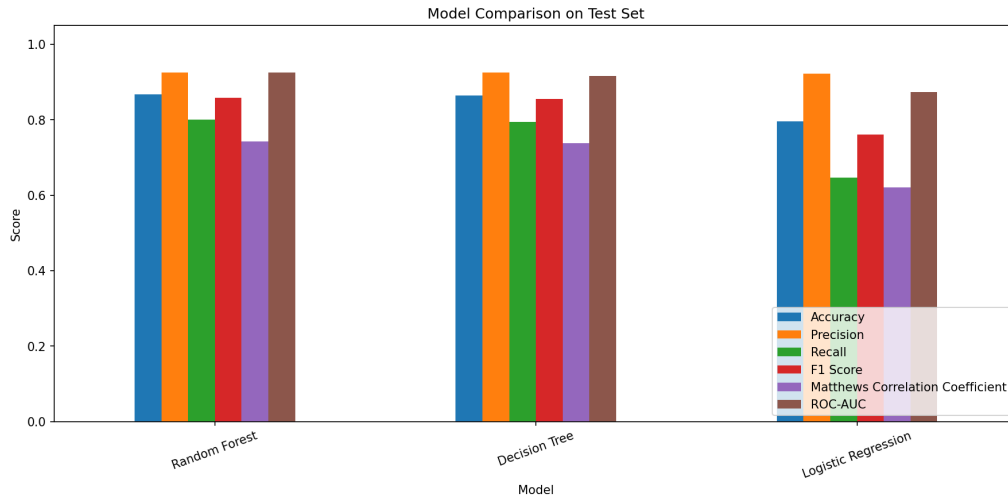


Figure 3: Comparison of model performance across evaluation metrics.

6.5 Precision-Recall Analysis

Precision-recall curves were used because phishing detection focuses strongly on the positive phishing class. PR-AUC is useful because it evaluates how well the model maintains precision while increasing recall.

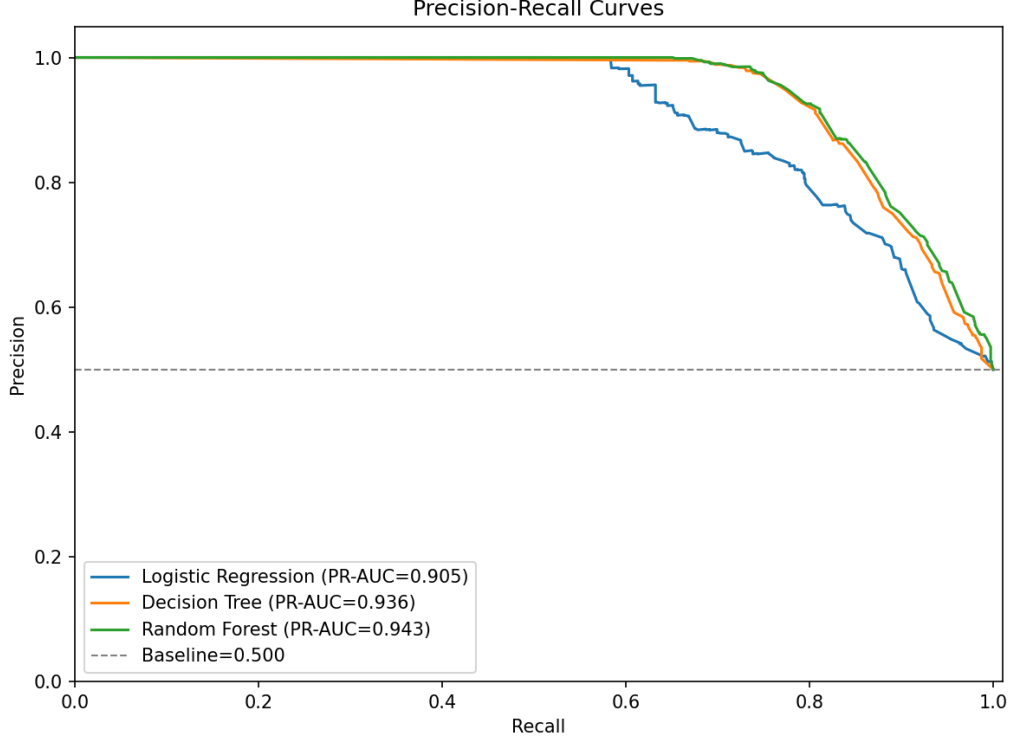


Figure 4: Precision-recall curves for the evaluated models.

Random Forest achieved the highest PR-AUC among the evaluated models, indicating strong performance for detecting phishing websites across different decision thresholds.

6.6 Duplicate Row Robustness Analysis

The dataset contains many duplicated rows. Exact duplicate rows may inflate performance if identical examples appear in both the training and testing sets. To assess this risk, the models were evaluated on both the original dataset and a deduplicated version of the dataset.

Table 2: Dataset size before and after duplicate removal.

Dataset	Rows	Duplicate Rows	Legitimate	Phishing
Original	10000	5626	5000	5000
Deduplicated	4374	0	529	3845

After duplicate removal, the dataset became highly imbalanced: only 529 legitimate rows remained, compared with 3,845 phishing rows. This shows that the duplicate issue is not only a matter of repeated samples, but also affects the class distribution.

Table 3: Model performance on original and deduplicated datasets using a random split.

Dataset	Model	Accuracy	Precision	Recall	F1	F2	ROC-AUC	PR-AUC
Original	Random Forest	0.868	0.926	0.800	0.858	0.822	0.926	0.943
Original	Decision Tree	0.865	0.925	0.794	0.855	0.817	0.917	0.936
Original	Logistic Regression	0.796	0.923	0.647	0.761	0.688	0.874	0.905
Deduplicated	Logistic Regression	0.902	0.901	0.997	0.947	0.977	0.905	0.986
Deduplicated	Random Forest	0.887	0.907	0.971	0.938	0.958	0.875	0.980
Deduplicated	Decision Tree	0.881	0.906	0.965	0.935	0.953	0.805	0.955

The deduplicated results are not directly comparable to the original results because duplicate removal changed the class balance substantially. The large reduction in legitimate samples indicates that duplicate records were disproportionately concentrated in that class, substantially changing the class distribution after duplicate removal.

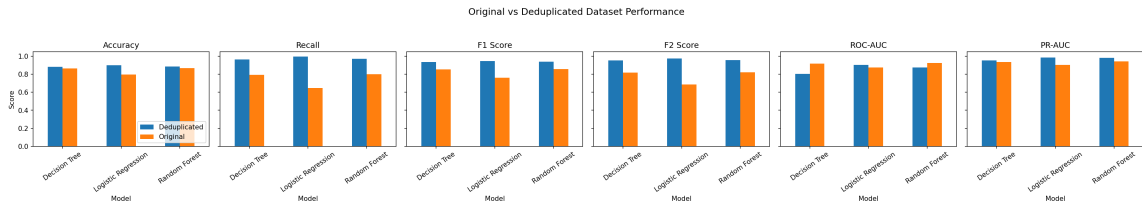


Figure 5: Comparison of original and deduplicated dataset performance.

6.7 Domain-Aware Generalization Analysis

To test whether models generalize to unseen domains, a grouped split by **Domain** was performed using **GroupShuffleSplit**. This ensures that the same domain cannot appear in both the training and testing sets.

Table 4: Grouped split summary.

Split	Rows	Unique Domains	Label 0 Rate	Label 1 Rate
Train	8205	2842	0.504	0.496
Test	1795	711	0.482	0.518

The overlap between training and testing domains was zero domains. This confirms that the grouped split prevents domain leakage and provides a stricter evaluation of generalization to unseen domains.

Table 5: Random split compared with grouped split by domain.

Split	Model	Accuracy	Precision	Recall	F1	F2	ROC-AUC	PR-AUC
Random Row Split	Random Forest	0.868	0.926	0.800	0.858	0.822	0.926	0.943
Random Row Split	Decision Tree	0.865	0.925	0.794	0.855	0.817	0.917	0.936
Random Row Split	Logistic Regression	0.796	0.923	0.647	0.761	0.688	0.874	0.905
Grouped by Domain	Decision Tree	0.771	0.771	0.792	0.782	0.788	0.825	0.839
Grouped by Domain	Random Forest	0.769	0.768	0.794	0.781	0.789	0.844	0.887
Grouped by Domain	Logistic Regression	0.802	0.924	0.672	0.778	0.711	0.848	0.899

The grouped split shows that Random Forest performance drops from F1 Score 0.858 to 0.781. This suggests that the regular random split may overestimate performance because related domains can appear in both training and testing sets. The grouped split is therefore a stricter and more realistic evaluation of generalization to unseen domains.

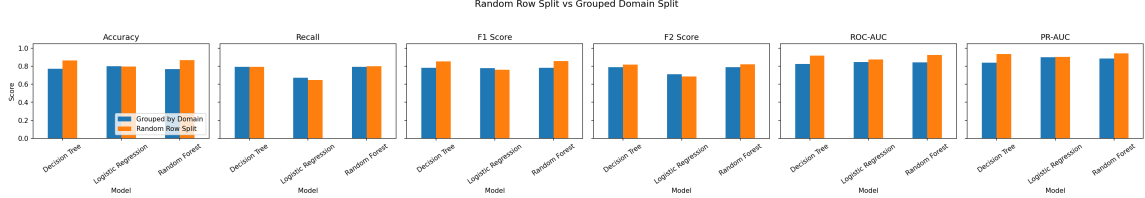


Figure 6: Random row split compared with grouped split by domain.

6.8 Feature Importance

Feature importance analysis was performed using the Random Forest classifier, which achieved the highest performance on the regular split.

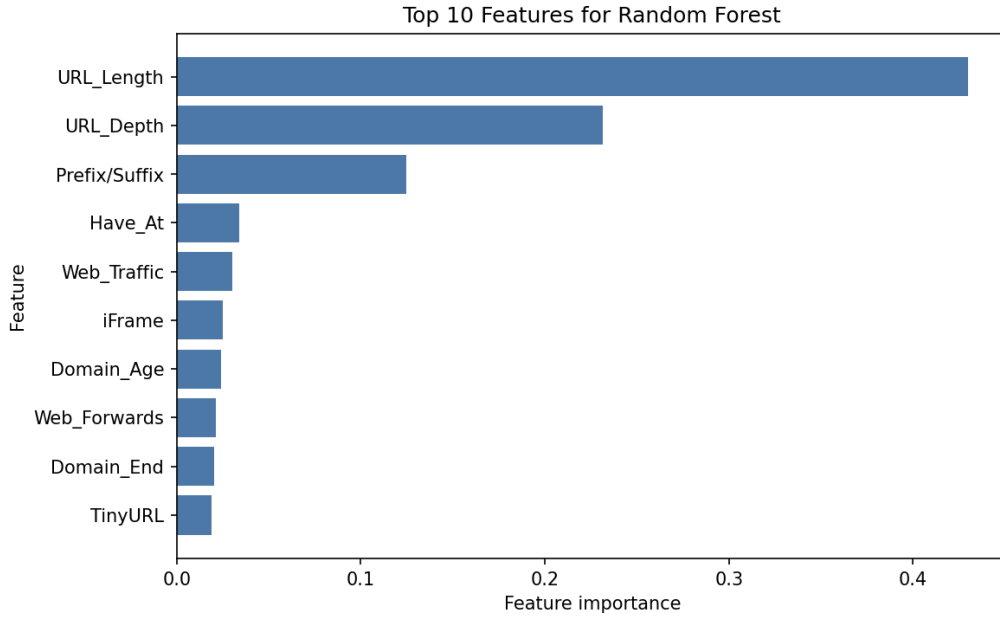


Figure 7: Top feature importance values for the Random Forest model.

As shown in Figure 7, URL Length and URL Depth are among the most influential features. This suggests that structural URL characteristics play an important role in distinguishing phishing websites from legitimate websites.

6.9 Threshold and Risk Trade-off Analysis

For the default Random Forest threshold of 0.5, the model produced 64 false positives and 200 false negatives. In phishing detection, false negatives are particularly important

because they represent phishing websites that were incorrectly classified as legitimate.

Threshold analysis was performed to study the trade-off between reducing false negatives and increasing false positives.

The main error pattern for the Random Forest model at the default threshold of 0.5 was the presence of 200 false negatives, indicating that some phishing websites were still classified as legitimate despite the model’s strong overall performance.

Table 6: Threshold analysis for the Random Forest model.

Threshold	Precision	Recall	F1	F2	FP	FN
0.2	0.704	0.928	0.801	0.873	390	72
0.3	0.863	0.843	0.853	0.847	134	157
0.4	0.904	0.815	0.857	0.831	87	185
0.5	0.926	0.800	0.858	0.822	64	200
0.6	0.978	0.741	0.843	0.779	17	259
0.7	0.985	0.730	0.839	0.770	11	270

Lowering the threshold from 0.5 to 0.2 reduced false negatives from 200 to 72, but increased false positives from 64 to 390. This demonstrates the operational trade-off in phishing detection: a lower threshold catches more phishing websites but creates more alerts for legitimate websites. Threshold 0.3 may represent a practical compromise, reducing false negatives from 200 to 157 while increasing false positives to only 134 instead of 390. Depending on the deployment context, this threshold may provide a better balance between detecting phishing websites and limiting unnecessary alerts.

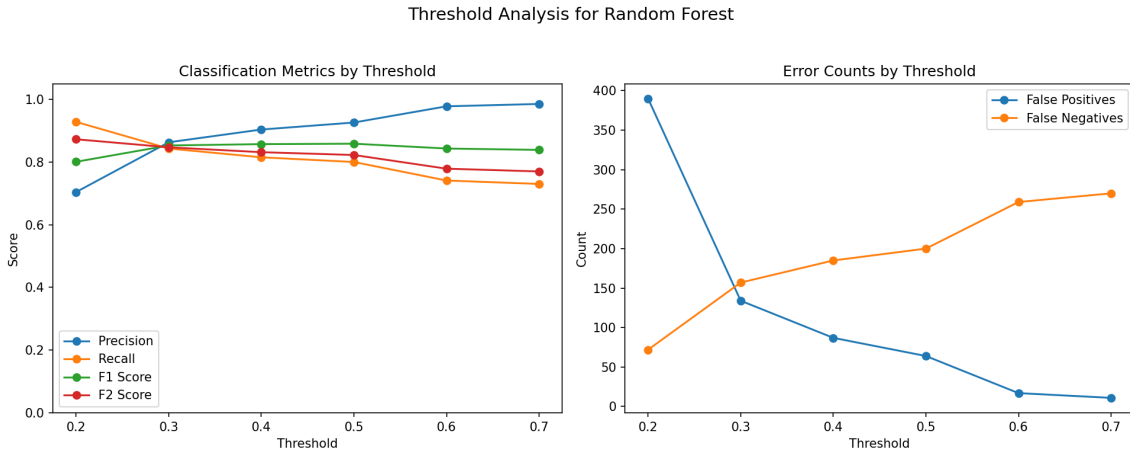


Figure 8: Threshold analysis for Random Forest.

7 Summing It Up

The selected GitHub repository, *Phishing Website Detection* by Gangesh Basker, addresses phishing website detection using supervised machine learning. The project uses

the `5.urldata.csv` dataset, which contains engineered URL and website-related features for classifying websites as legitimate or phishing.

The reproduction confirms that the proposed approach can achieve strong results on the provided dataset. Random Forest performed best on the regular random split, suggesting that ensemble tree-based methods are well suited to this tabular feature-based phishing detection task.

However, the deeper analysis shows that the original claim should be interpreted carefully. The dataset contains many duplicate rows, and grouped splitting by domain produces lower performance. This means that the original evaluation may overestimate how well the model generalizes to truly unseen websites.

Based on the results, Random Forest is the strongest choice among the three evaluated models for the regular random split, but it should be used with caution. In a practical cybersecurity setting, the model should not be deployed without duplicate-aware preprocessing, domain-aware validation, evaluation on more recent data, and threshold tuning based on operational security requirements.

The main recommendation is therefore to treat this project as a useful baseline rather than a deployment-ready phishing detection system. The results demonstrate that engineered features and classical machine learning can be effective, but they also show that evaluation design strongly affects the conclusions. A reliable phishing detection system must combine predictive performance with careful validation, interpretability, reproducibility, and awareness of real-world threat conditions.

8 Conclusion

This project reproduced and extended a phishing website detection workflow using classical machine learning techniques. The dataset was inspected, analyzed, preprocessed, and used to train Logistic Regression, Decision Tree, and Random Forest classifiers.

The reproduction supports the original author’s general claim that engineered URL and website-related features can be effective for phishing detection. On the regular random split, Random Forest achieved the strongest performance with F1 Score 0.858, F2 Score 0.822, ROC-AUC 0.926, and PR-AUC 0.943.

At the same time, the robustness analyses show that the initial performance estimates require careful interpretation. Removing duplicate rows substantially changed the dataset size and class balance, and grouped splitting by domain reduced Random Forest performance. These findings indicate that duplicate rows and domain leakage can noticeably affect evaluation reliability and may lead to overly optimistic estimates of real-world performance.

The threshold analysis also shows that phishing detection should not rely only on the default 0.5 classification threshold. Lowering the threshold can substantially reduce false

negatives, but at the cost of more false positives. Therefore, the best threshold depends on the operational risk tolerance of the cybersecurity system.

The main lesson from this project is that strong benchmark results are not enough on their own. In cybersecurity applications, evaluation methodology is especially important because attackers constantly change their behavior, and models that perform well on historical data may not generalize to new phishing campaigns. The project also shows the importance of checking data quality, preventing leakage, and choosing metrics that reflect the real cost of different error types.

The main limitations of this reproduction are that it uses a single dataset, relies only on engineered tabular features, and evaluates only Logistic Regression, Decision Tree, and Random Forest. No temporal split was possible because the dataset does not include a temporal feature, and the results may not fully reflect performance against newer phishing techniques.

Future improvements could include cross-validation, evaluation on more recent phishing datasets, richer domain- and content-based features, modern ensemble or deep learning models, and threshold selection based on explicit operational security costs. In addition, duplicate-aware preprocessing and domain-aware validation should be considered standard practice when evaluating phishing detection systems to obtain more realistic estimates of real-world performance.

Ultimately, robust evaluation practices are essential for developing trustworthy machine learning systems for cybersecurity applications.

9 References

1. Gangesh Basker. *Phishing Website Detection Repository*. <https://github.com/gangeshbaskerr/Phishing-Website-Detection>
2. Scikit-learn Documentation. <https://scikit-learn.org/>
3. Pandas Documentation. <https://pandas.pydata.org/>